

001_understanding-data

May 11, 2026

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: df = pd.read_csv("../dataset/train.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7613 entries, 0 to 7612
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   id          7613 non-null   int64
 1   keyword     7552 non-null   object
 2   location    5080 non-null   object
 3   text        7613 non-null   object
 4   target      7613 non-null   int64
dtypes: int64(2), object(3)
memory usage: 297.5+ KB
```

```
[3]: df.shape
```

```
[3]: (7613, 5)
```

```
[4]: df.sample(5)
```

```
[4]:      id      keyword      location \
2256  3235      deluged              NaN
983   1422  body%20bag      New York
1506  2174  catastrophic  Inexpressible Island
3591  5130      fatal              Thane
2125  3053      deaths              NaN

      text  target
2256  Businesses are deluged with inroices.|Make you...    0
983   New Ladies Shoulder Tote Handbag Faux Leather ...    0
1506  The Catastrophic Effects of Hiroshima and Naga...    1
3591  11-Year-Old Boy Charged With Manslaughter of T...    1
```

```
2125 #vaxshill 2 deaths from measles complications ... 1
```

Check NULL value rows

```
[5]: columns_list = df.columns

for col in columns_list:
    print(f"Column name: {col} contains {df[col].isnull().sum()} NULL entries.")
```

```
Column name: id contains 0 NULL entries.
Column name: keyword contains 61 NULL entries.
Column name: location contains 2533 NULL entries.
Column name: text contains 0 NULL entries.
Column name: target contains 0 NULL entries.
```

```
[6]: df['keyword'].value_counts()
```

```
[6]: keyword
fatalities          45
deluge              42
armageddon          42
sinking            41
damage             41
..
forest%20fire       19
epicentre           12
threat              11
inundation          10
radiation%20emergency 9
Name: count, Length: 221, dtype: int64
```

```
[7]: df['location'].value_counts()
```

```
[7]: location
USA          104
New York     71
United States 50
London       45
Canada       29
...
Montr  al, Qu  bec 1
Montreal          1
  T: 6.4682,3.18287 1
Live4Heed??       1
Lincoln           1
Name: count, Length: 3341, dtype: int64
```

Conclusion Delete columns **location**, In my opinion, there is no usage of this column specially when classifying for the *Disaster*. **keyword** may helps, but there are also so rows which contains

missing values. For that we'll fill those value using values which are occuring the most (*mode* value). Final deletion, **id** column, totally crap no use at all.

```
[8]: new_df = df.drop(columns=['location', 'id'])
     new_df.shape
```

```
[8]: (7613, 3)
```

```
[9]: KEYWORD_FILLER=new_df['keyword'].mode().values[0]

     print("Previously: {}".format(new_df['keyword'].isnull().sum()))
     new_df.fillna({'keyword': KEYWORD_FILLER}, inplace=True)
     print("Now: {}".format(new_df['keyword'].isnull().sum()))
```

Previously: 61

Now: 0

```
[10]: for col in new_df.columns:
       print(
           f"Column: {col} has {new_df[col].isnull().sum()} NULL values"
       )
```

Column: keyword has 0 NULL values

Column: text has 0 NULL values

Column: target has 0 NULL values

```
[11]: new_df.sample(5)
```

```
[11]:
```

	keyword	text	target
2696	detonation	Detonation into the realistic assets entering ...	0
2689	detonation	Do you want to play a game?\nhttp://t.co/sQFp6...	0
5555	rainstorm	major rainstorm happening! I'm gonna lie down ...	1
2482	desolate	The once desolate valley was transformed into ...	0
758	blew%20up	Max blew tf up ! ?????? shots fired ???? #Catf...	0

Clean the data

```
[12]: # import nltk
     # nltk.download('popular')
```

```
[13]: import re
     from nltk.tokenize import word_tokenize, sent_tokenize
```

```
[14]: def remove_punctuation_and_number(text):
       cleaned_text = re.sub(r'[\W\s]', '', text)
       cleaned_text = re.sub(r'[\n\t\r\f\v\b\A]', '', cleaned_text)
       return cleaned_text
```

```
[15]: remove_punctuation_and_number("Hii there !!!, 9090_ditto that's me cheetos,↵
      ↪here is $20")
```

```
[15]: 'Hii there 9090_ditto thats me cheetos here is 20'
```

```
[16]: print(word_tokenize("hello world, this is me"))
      print(sent_tokenize("hii, julie hello, ben"))
```

```
['hello', 'world', ',', 'this', 'is', 'me']
['hii, julie hello, ben']
```

```
[17]: def clean_data(text: str) -> list:

      lower_case_text = text.lower()
      cleaned_text = remove_punctuation_and_number(lower_case_text)
      return cleaned_text
```

```
[18]: new_df["complete_text"] = new_df['keyword'] + " " + new_df['text']
      new_df.sample(1)
```

```
[18]:      keyword                                text  target \
1898  crushed  Jesus Christ that ball was fucking crushed!! #...      0

      complete_text
1898  crushed Jesus Christ that ball was fucking cru...
```

```
[19]: new_df.drop(columns=['text', 'keyword'], inplace=True)
```

```
[20]: new_df
```

```
[20]:      target                                complete_text
0          1  fatalities Our Deeds are the Reason of this #e...
1          1  fatalities Forest fire near La Ronge Sask. Canada
2          1  fatalities All residents asked to 'shelter in ...
3          1  fatalities 13,000 people receive #wildfires ev...
4          1  fatalities Just got sent this photo from Ruby ...
...      ...
7608       1  fatalities Two giant cranes holding a bridge c...
7609       1  fatalities @aria_ahrany @TheTawniest The out o...
7610       1  fatalities M1.94 [01:04 UTC]?5km S of Volcano ...
7611       1  fatalities Police investigating after an e-bik...
7612       1  fatalities The Latest: More Homes Razed by Nor...

[7613 rows x 2 columns]
```

```
[21]: new_df['complete_text'] = new_df['complete_text'].apply(lambda x: clean_data(x))
```

```
[22]: new_df.sample(1)
```

```
[22]:      target                                complete_text
      958          0  body20bag louis vuitton monogram sophie limite...
```

```
[23]: new_df.to_csv("../dataset/updated_train.csv")
```

Understanding testing dataset file

```
[24]: test_file = pd.read_csv("../dataset/test.csv")
      test_file.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3263 entries, 0 to 3262
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   id          3263 non-null   int64
 1   keyword     3237 non-null   object
 2   location    2158 non-null   object
 3   text        3263 non-null   object
dtypes: int64(1), object(3)
memory usage: 102.1+ KB
```

```
[25]: test_file.drop(columns=['location', 'id'], inplace=True)
      test_file.shape
```

```
[25]: (3263, 2)
```

```
[26]: test_file.fillna({
      'keyword': KEYWORD_FILLER
    }, inplace=True)
```

```
[28]: test_file["complete_text"] = test_file['keyword'] + " " + test_file['text']
      test_file.drop(columns=['keyword', 'text'], inplace=True)
      test_file['complete_text'] = test_file['complete_text'].apply(lambda x:
      ↪ clean_data(x))
```

```
[30]: test_file.to_csv("../dataset/updated_test.csv", index=False)
```

```
[ ]:
```

002_model_building

May 11, 2026

```
[39]: import torch
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from transformers import AutoTokenizer
from transformers import AutoModelForSequenceClassification
```

```
[40]: DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
# MODEL_NAME = "distilbert-base-uncased"
MODEL_NAME = "roberta-base"
```

```
[41]: tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

```
config.json: 0%|          | 0.00/481 [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/25.0 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
[42]: def preprocess_fxn(text):
    return tokenizer(text, truncation=True, padding=True, max_length=256)
```

```
[43]: data = pd.read_csv("./updated_train.csv")
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7613 entries, 0 to 7612
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Unnamed: 0      7613 non-null   int64
 1   target          7613 non-null   int64
 2   complete_text   7613 non-null   object
dtypes: int64(2), object(1)
memory usage: 178.6+ KB
```

```
[44]: data.drop(columns=['Unnamed: 0'], inplace=True)
data.sample(2)
```

```
[44]:      target      complete_text
2644      1  destruction so you have a new weapon that can ...
2227      0  deluge the famping things i do for gishwhes ju...
```

```
[45]: texts = data['complete_text'].tolist()
labels = data['target'].tolist()
```

```
[46]: max(
    [len(review) for review in texts]
)
```

```
[46]: 159
```

```
[47]: from sklearn.model_selection import train_test_split

train_texts, val_texts, train_labels, val_labels = train_test_split(
    texts,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)
```

```
[48]: train_encoding = tokenizer(train_texts, truncation=True, padding=True,
    ↪max_length=256)
val_encoding = tokenizer(val_texts, truncation=True, padding=True,
    ↪max_length=256)
```

```
[49]: from torch.utils.data import Dataset
```

```
[50]: class TextDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        item = {
            key: torch.tensor(val[idx]) for key, val in self.encodings.items()
        }

        item['labels'] = torch.tensor(self.labels[idx])
```

```
return item
```

```
[51]: train_dataset = TextDataset(train_encoding, train_labels)
      val_dataset = TextDataset(val_encoding, val_labels)
```

```
[52]: model = AutoModelForSequenceClassification.from_pretrained(
      MODEL_NAME,
      num_labels=2
    )

    model = model.to(DEVICE)
```

```
model.safetensors:  0%|          | 0.00/499M [00:00<?, ?B/s]
```

```
Loading weights:  0%|          | 0/197 [00:00<?, ?it/s]
```

RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.out_proj.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
[53]: import numpy as np
      from sklearn.metrics import accuracy_score, f1_score

      def compute_metrics(eval_pred):
          logits, labels = eval_pred
          predictions = np.argmax(logits, axis=-1)

          return {
              "accuracy": accuracy_score(labels, predictions),
              "f1": f1_score(labels, predictions)
          }
```



```
[56]: from transformers import TrainingArguments, Trainer
      from transformers import DataCollatorWithPadding
      from transformers import EarlyStoppingCallback

      data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

      training_args = TrainingArguments(
          output_dir="./results",
          num_train_epochs=15,
          lr_scheduler_type="cosine",
          per_device_train_batch_size=16,
          weight_decay=0.01,
          warmup_ratio=0.1,
          per_device_eval_batch_size=8,
          warmup_steps=100,
          logging_dir="./logs",
          logging_steps=10,
          fp16=True,
          eval_strategy="epoch",
          save_strategy="epoch",
          load_best_model_at_end=True,
          metric_for_best_model="eval_loss"
      )
```

warmup_ratio is deprecated and will be removed in v5.2. Use `warmup\_steps` instead.

`logging\_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD\_LOGGING\_DIR` instead.

```
[57]: trainer = Trainer(
      model=model,
      args=training_args,
      train_dataset=train_dataset,
      eval_dataset=val_dataset,
      data_collator=data_collator,
      compute_metrics=compute_metrics,
      callbacks=[EarlyStoppingCallback(early_stopping_patience=3)]
  )
```

```
[58]: trainer.train()
```

<IPython.core.display.HTML object>

```
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
```

Writing model shards: 0%| | 0/1 [00:00<?, ?it/s]

Writing model shards: 0%| | 0/1 [00:00<?, ?it/s]

There were missing keys in the checkpoint model loaded:

['roberta.embeddings.LayerNorm.weight', 'roberta.embeddings.LayerNorm.bias',
'roberta.encoder.layer.0.attention.output.LayerNorm.weight',
'roberta.encoder.layer.0.attention.output.LayerNorm.bias',
'roberta.encoder.layer.0.output.LayerNorm.weight',
'roberta.encoder.layer.0.output.LayerNorm.bias',
'roberta.encoder.layer.1.attention.output.LayerNorm.weight',
'roberta.encoder.layer.1.attention.output.LayerNorm.bias',
'roberta.encoder.layer.1.output.LayerNorm.weight',
'roberta.encoder.layer.1.output.LayerNorm.bias',
'roberta.encoder.layer.2.attention.output.LayerNorm.weight',
'roberta.encoder.layer.2.attention.output.LayerNorm.bias',
'roberta.encoder.layer.2.output.LayerNorm.weight',
'roberta.encoder.layer.2.output.LayerNorm.bias',
'roberta.encoder.layer.3.attention.output.LayerNorm.weight',
'roberta.encoder.layer.3.attention.output.LayerNorm.bias',
'roberta.encoder.layer.3.output.LayerNorm.weight',
'roberta.encoder.layer.3.output.LayerNorm.bias',
'roberta.encoder.layer.4.attention.output.LayerNorm.weight',
'roberta.encoder.layer.4.attention.output.LayerNorm.bias',
'roberta.encoder.layer.4.output.LayerNorm.weight',
'roberta.encoder.layer.4.output.LayerNorm.bias',
'roberta.encoder.layer.5.attention.output.LayerNorm.weight',
'roberta.encoder.layer.5.attention.output.LayerNorm.bias',
'roberta.encoder.layer.5.output.LayerNorm.weight',
'roberta.encoder.layer.5.output.LayerNorm.bias',
'roberta.encoder.layer.6.attention.output.LayerNorm.weight',
'roberta.encoder.layer.6.attention.output.LayerNorm.bias',
'roberta.encoder.layer.6.output.LayerNorm.weight',
'roberta.encoder.layer.6.output.LayerNorm.bias',
'roberta.encoder.layer.7.attention.output.LayerNorm.weight',
'roberta.encoder.layer.7.attention.output.LayerNorm.bias',
'roberta.encoder.layer.7.output.LayerNorm.weight',
'roberta.encoder.layer.7.output.LayerNorm.bias',
'roberta.encoder.layer.8.attention.output.LayerNorm.weight',
'roberta.encoder.layer.8.attention.output.LayerNorm.bias',
'roberta.encoder.layer.8.output.LayerNorm.weight',
'roberta.encoder.layer.8.output.LayerNorm.bias',
'roberta.encoder.layer.9.attention.output.LayerNorm.weight',
'roberta.encoder.layer.9.attention.output.LayerNorm.bias',
'roberta.encoder.layer.9.output.LayerNorm.weight',
'roberta.encoder.layer.9.output.LayerNorm.bias',
'roberta.encoder.layer.10.attention.output.LayerNorm.weight',
'roberta.encoder.layer.10.attention.output.LayerNorm.bias',
'roberta.encoder.layer.10.output.LayerNorm.weight',

```

'roberta.encoder.layer.10.output.LayerNorm.bias',
'roberta.encoder.layer.11.attention.output.LayerNorm.weight',
'roberta.encoder.layer.11.attention.output.LayerNorm.bias',
'roberta.encoder.layer.11.output.LayerNorm.weight',
'roberta.encoder.layer.11.output.LayerNorm.bias'].
There were unexpected keys in the checkpoint model loaded:
['roberta.embeddings.LayerNorm.beta', 'roberta.embeddings.LayerNorm.gamma',
'roberta.encoder.layer.0.attention.output.LayerNorm.beta',
'roberta.encoder.layer.0.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.0.output.LayerNorm.beta',
'roberta.encoder.layer.0.output.LayerNorm.gamma',
'roberta.encoder.layer.1.attention.output.LayerNorm.beta',
'roberta.encoder.layer.1.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.1.output.LayerNorm.beta',
'roberta.encoder.layer.1.output.LayerNorm.gamma',
'roberta.encoder.layer.2.attention.output.LayerNorm.beta',
'roberta.encoder.layer.2.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.2.output.LayerNorm.beta',
'roberta.encoder.layer.2.output.LayerNorm.gamma',
'roberta.encoder.layer.3.attention.output.LayerNorm.beta',
'roberta.encoder.layer.3.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.3.output.LayerNorm.beta',
'roberta.encoder.layer.3.output.LayerNorm.gamma',
'roberta.encoder.layer.4.attention.output.LayerNorm.beta',
'roberta.encoder.layer.4.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.4.output.LayerNorm.beta',
'roberta.encoder.layer.4.output.LayerNorm.gamma',
'roberta.encoder.layer.5.attention.output.LayerNorm.beta',
'roberta.encoder.layer.5.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.5.output.LayerNorm.beta',
'roberta.encoder.layer.5.output.LayerNorm.gamma',
'roberta.encoder.layer.6.attention.output.LayerNorm.beta',
'roberta.encoder.layer.6.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.6.output.LayerNorm.beta',
'roberta.encoder.layer.6.output.LayerNorm.gamma',
'roberta.encoder.layer.7.attention.output.LayerNorm.beta',
'roberta.encoder.layer.7.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.7.output.LayerNorm.beta',
'roberta.encoder.layer.7.output.LayerNorm.gamma',
'roberta.encoder.layer.8.attention.output.LayerNorm.beta',
'roberta.encoder.layer.8.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.8.output.LayerNorm.beta',
'roberta.encoder.layer.8.output.LayerNorm.gamma',
'roberta.encoder.layer.9.attention.output.LayerNorm.beta',
'roberta.encoder.layer.9.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.9.output.LayerNorm.beta',
'roberta.encoder.layer.9.output.LayerNorm.gamma',
'roberta.encoder.layer.10.attention.output.LayerNorm.beta',

```

```
'roberta.encoder.layer.10.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.10.output.LayerNorm.beta',
'roberta.encoder.layer.10.output.LayerNorm.gamma',
'roberta.encoder.layer.11.attention.output.LayerNorm.beta',
'roberta.encoder.layer.11.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.11.output.LayerNorm.beta',
'roberta.encoder.layer.11.output.LayerNorm.gamma'].
```

```
[58]: TrainOutput(global_step=2286, training_loss=0.3424704767900502,
metrics={'train_runtime': 456.4207, 'train_samples_per_second': 200.144,
'train_steps_per_second': 12.521, 'total_flos': 1408312201590000.0,
'train_loss': 0.3424704767900502, 'epoch': 6.0})
```

```
[59]: trainer.evaluate()
```

```
<IPython.core.display.HTML object>
```

```
[59]: {'eval_loss': 0.3982447385787964,
'eval_accuracy': 0.8483256730137886,
'eval_f1': 0.8156424581005587,
'eval_runtime': 2.9497,
'eval_samples_per_second': 516.324,
'eval_steps_per_second': 64.752,
'epoch': 6.0}
```

```
[ ]:
```

```
[63]: test_data = pd.read_csv("./updated_test.csv")
test_data.info()

test_samples = test_data['complete_text'].tolist()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3263 entries, 0 to 3262
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   complete_text    3263 non-null   object
dtypes: object(1)
memory usage: 25.6+ KB
```

```
[61]: # text = "This movie was amazing!"
```

```
inputs = tokenizer(
    test_samples,
    return_tensors="pt",
    truncation=True,
    padding=True
).to(DEVICE)
```

```
with torch.no_grad():
    outputs = model(**inputs)

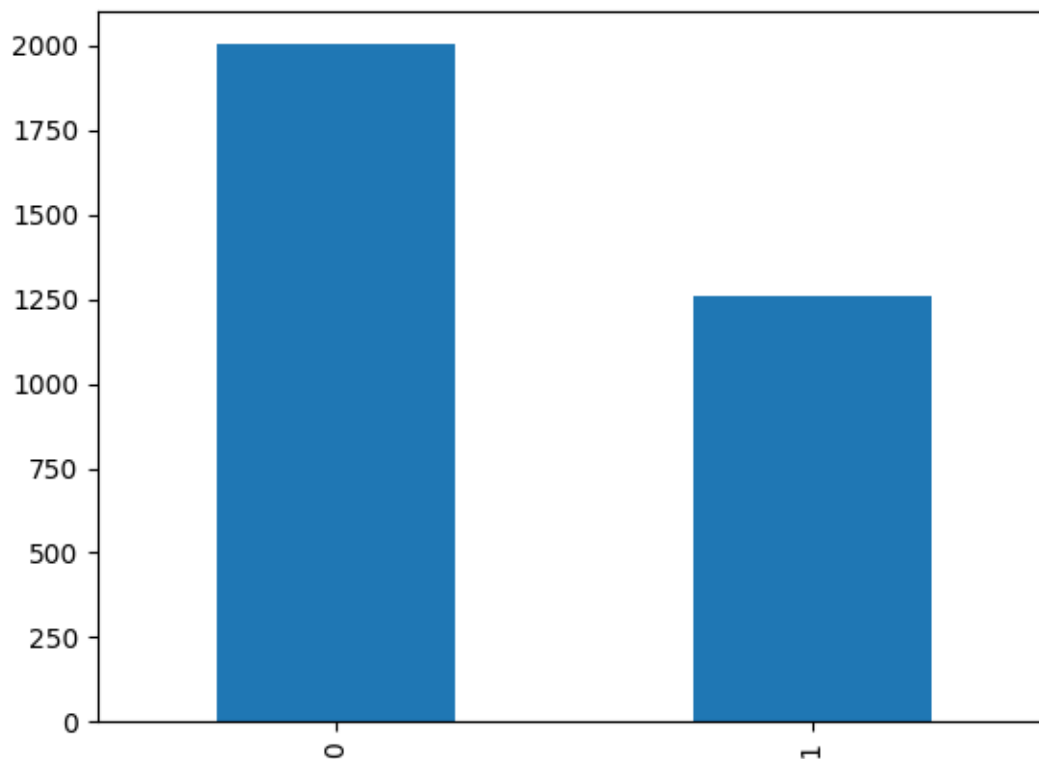
prediction = torch.argmax(outputs.logits, dim=1)

print(prediction)
```

```
tensor([1, 1, 1, ..., 1, 1, 1], device='cuda:0')
```

```
[62]: pd.Series(prediction.tolist()).value_counts().plot(kind='bar')
print(pd.Series(prediction.tolist()).value_counts())
```

```
0    2002
1    1261
Name: count, dtype: int64
```



```
[ ]:
```